



ArkTS code generation: A comprehensive evaluation with large language models

Ekin Can Erkuş¹ · Cansen Çağlayan Yılmaz¹ · Mustafa Umut Demirezen¹

Received: 15 April 2025 / Accepted: 6 March 2026 / Published online: 2 April 2026
© The Author(s) 2026

Abstract

This paper presents the first systematic evaluation of ArkTS code generation with large language models, which uses 300 prompts across three difficulty levels and measures Pass@1, compilation rate, and generation time in milliseconds while it maps compiler messages into syntax, type, undefined reference, and other failures, and it also adds an independent LLM judge with a fixed scoring rule. We evaluate 21 models and we find that functional correctness stays low and compilation varies widely, since DeepSeek-R1 reaches 22.7% Pass@1, Claude-3.7-Sonnet reaches 13.7%, and Gemini-2.0-Pro-Experimental-02-05 reaches 12.5%, while several widely used systems return no correct ArkTS solutions at Pass@1. Performance drops from Easy to Hard and drops again when we move from algorithms and data structures to API usage and UI design, which shows that reactive state and lifecycle handling remain difficult, and lower syntax error rates often come with many undefined references and type mismatches which keep Pass@1 low even when code compiles, and short generation time does not guarantee higher compilation success. A small held out study which adds one compact ArkTS example with explicit imports and types and then applies a single compiler guided repair step raises Pass@1 from 23.3% to 36.7% with a modest latency cost, and the largest gains come from fewer missing imports, clearer public signatures, and quick fixes to bracket balance and unresolved symbols. These observations are simple and they point to practical gaps that need patient work on ArkTS typing, imports, and lifecycle usage before code generation becomes more dependable.

Keywords ArkTS · HarmonyOS · Automated code generation · Large language models · Evaluation

Ekin Can Erkuş, Cansen Çağlayan Yılmaz and Mustafa Umut Demirezen contributed equally to this work.
Communicated by: Meiyappan Nagappan.

Extended author information available on the last page of the article

1 Introduction

Large language models (LLMs) now generate code with growing accuracy in mainstream languages, including Python, Java, and JavaScript, and do so mainly because training data are rich and the tooling is mature (Jiang et al. 2024). Many real projects also use languages that are less common and more domain-specific, which remain underexplored, such as ArkTS for HarmonyOS (Zhou et al. 2025). These languages bring their own type rules, libraries, and idioms which current models rarely see during training, and this gap often leads to code that does not compile or runs as expected (Chen et al. 2025).

ArkTS (ArkUI TypeScript) extends TypeScript with a stronger type system and with reactive UI and state patterns for HarmonyOS, making it attractive for mobile and device interfaces (Zhou et al. 2025). ArkTS looks familiar to web developers, yet it adds language forms and component lifecycles that require precise code that many models do not yet learn well (Li et al. 2023). Recent studies on low resource and domain specific languages also show that targeted fine tuning and prompt augmentation help when training data are scarce (Lin et al. 2025). Although ArkTS promises rapid interface development and robust application design, preliminary evaluations reveal that even when LLMs produce syntactically correct code for common languages, their performance on ArkTS is inconsistent, primarily due to limited representation during training (Chen et al. 2025).

Empirical studies indicate that fine-tuning LLMs in code-specific data sets boosts functional correctness on benchmarks like HumanEval (Jiang et al. 2024). This progress comes mainly from advances in Transformer models, which include multiheaded attention that captures long-range relations, and from additional techniques that include context learning, prompt design, and the use of semantic metadata with carefully curated examples (Yenduri et al. 2024). In addition, parameter efficient fine tuning methods such as Quantized Low-Rank Adaptation adapt general purpose models to domain specific tasks at lower cost, which consistently increases BLEU scores and reduces syntax errors (Weyssow et al. 2023).

The literature reports many ideas for automated code generation which cover planning, search, and repair, and self planning methods that separate the planning and implementation phases frequently improve Pass@1 with relative gains up to 25.4% (Jiang et al. 2023). Additional debugging methods that use print statements to track execution and fix latent errors also improve code quality (Hu et al. 2024). Guided planning decoding combines look-ahead search with informed token choice to produce higher quality output (Zhang et al. 2023). Work on shifts in training data (Arakelyan et al. 2023) and on automated test case creation (Li and Yuan 2024) supports a larger evaluation with metrics that include Pass@1 and Pass@n that provide fine-grained findings about model behavior (Yeo et al. 2024). Preliminary studies reveal a marked gap between the amount of code that models produce and its functional correctness; for example, DeepSeek-R1, Claude 3.7 Sonnet, and Gemini 2.0 Pro Experimental 02-05 reach Pass@1 scores of 22.7%, 13.7%, and 12.5% respectively, while GPT 4 and several Qwen variants fail to produce compilable ArkTS code (Jiang et al. 2023; Hu et al. 2024).

These planning and debugging studies also report better readability and maintainability because the process becomes clear and incremental (Jiang et al. 2023; Hu et al. 2024). Guided planning decoding applies lookahead search which generates candidate programs

and then refines token choices with public tests (Zhang et al. 2023), and adaptive temperature sampling adjusts randomness between hard and easy tokens which steadily improves output quality (Zhu et al. 2023). When these methods pair with evaluation metrics that include Pass@1 and Pass@n, they allow performance to be judged more fairly across settings that sit outside a model's main domain (Yeo et al. 2024), and many agent workflows and process models, such as FlowGen (Lin et al. 2024), closely follow software development steps and support collaborative code generation which therefore narrows the gap between academic work and real world use.

1.1 Motivation

Motivated by these advances and by clear gaps for domain specific languages, this paper presents the first full evaluation framework for ArkTS code generation, and our main contribution is a single ArkTS benchmark which we carefully build and fully document for fair and repeatable evaluation; it contains 300 prompts that span easy, medium, and hard levels which cover algorithms, data structures, API usage, and UI patterns, and each prompt includes a clear problem statement and input and output contracts and at least one validated ArkTS reference solution, while the construction pipeline uses LLM assisted translation which human annotators refine with step by step rules that focus on ArkTS syntax, typing, and idioms, and we record all decisions with versioned templates and we balance sampling across categories and difficulty with fixed quotas and shuffled seeds, and we provide scripts which compile, test, and score code with Pass@1, compilation checks, and generation time in milliseconds, and we also release a compact table of dataset statistics which reports counts and lengths per category and per difficulty so that users can quickly understand coverage and reuse the set in a consistent way.

ArkTS sits in a low resource space which current models rarely see during training, and many generated outputs either do not compile or do not behave as intended, which together creates a need for a focused and balanced benchmark that measures both syntax and behavior in ways that closely match everyday development. Converting well known problems from HumanEvalX, APPS, and MBPP into ArkTS is a practical first step which preserves core logic and control flow while it deliberately stresses typing, imports, and component lifecycles, and mobile specific API usage and UI lifecycles sometimes require tasks that general benchmarks do not cover, which we add by carefully sampling from ArkTS repositories with tests and by keeping the same validation rules so that later waves remain directly comparable.

We measure Pass@1, the compilation rate, and generation time in milliseconds which we record strictly around model calls, and we then parse compiler logs with a compact taxonomy that counts syntax errors, type errors, and undefined references, and we also add a brief yet careful qualitative read of structure and ArkTS style so that failure modes become clearer, while an independent LLM judge scores the code from 1 to 10 with a fixed prompt and a three vote setting which steadily reduces variability, and we finally report its agreement with human labels. We also report per difficulty and per category breakdowns, and we include a dedicated threats to validity section which discusses toolchain pinning, sampling balance, judge stability, and manual labeling choices, while we briefly study few shot prompting and self-refinement as auxiliary setups which may reduce common errors.

1.2 Research Questions and Paper Structure

We organize the paper around four research questions which match the benchmark design, the evaluation pipeline, and the analyses that follow. Table 1 points to the main artifacts so that the reader can quickly locate the construction steps, the two-tier evaluation view, the pilot agent-style result, and the broader methodological outlook.

We answer the questions below.

- **RQ1:** *How often do LLMs compile and pass ArkTS tests?*
We define Pass@1, compilation rate, and generation time in Section 2 under *Metrics Analysis*, we report the aggregate outcomes in Section 3.1 with Table 8 for model ranking and to quantify uncertainty with Wilson intervals in Table 9, we relate timing and compilation in Fig. 4, and we give the exact throughput formula in (1) while *Environment and Testing Pipeline* clarifies coverage via *Total Generated* and the measurement window for latency.
- **RQ2:** *How do results vary by difficulty and by category?*
We slice performance by easy, medium, and hard in Section 3.2 with Table 11, we break down outcomes by Algorithms, Data Structures, API Usage, and UI Design in Table 12, and we read the trends together with the dataset design in Section 2 *Design Principles* so that the quota balance and the intended coverage are clear.
- **RQ3:** *What failure types dominate and how do they align with judge scores?*
We define the compiler error taxonomy in Table 6 in Section 2, we quantify failure mixes in Section 3.4 with Table 13, we track difficulty trends in Fig. 5, and we relate these patterns to the independent LLM judge in Fig. 2 and Table 7, while Appendix A.2 lists the exact diagnostic patterns and Appendix A.3 gives the rubric and the scoring protocol.
- **RQ4:** *What is the relation between speed and accuracy across models?*
We formalize solved per minute in (1) and report per-model values with uncertainty in Table 9 inside Section 3.1, we place average generation time against compilation success in Section 3.3 with Fig. 4, and we specify timing boundaries and measurement choices in Section 2 so that the trade off reads consistently across systems.

Across models, functional correctness stays low even when prompts compile, which points to gaps in ArkTS knowledge and in handling of framework lifecycles. These results, the details of the data set, and the error taxonomy appear in Sections 2 and 3; the extended

Table 1 Navigation guide to the main artifacts in the paper

Item	Location in the paper
Benchmark construction pipeline	Section 2.2 and Fig. 1.
Two-tier evaluation view (compile vs. tests)	Section 2 under Metrics Analysis and Table 2.
Tier outcome decomposition (Tier 1 vs. Tier 2 rates)	Section 3.1 and Table 10, plus Fig. 3.
Error taxonomy and difficulty trends	Tables 6, 13, and Fig. 5.
Pilot agent-style method with compiler feedback	Section 3.5 and Table 14.
Broader outlook beyond ArkTS	Section 4.3.

discussion with practical guidance and broader lessons appears in Section 4; and new sections for threats to validity, limitations, and greater impact also appear in Section 4, while Section 5 closes with the next steps that we plan for data, prompts, and evaluation design.

Later sections also describe how small ArkTS exemplars and a short compiler guided repair step change behavior. Section 2.3 explains how we construct these compact examples, and Section 3.5 reports a small held out study that uses structured input and a single repair step which behaves like a simple process based method.

To make the revised material easy to trace, Section 2 introduces the two stage view of compilation and tests and the error taxonomy in Tables 2 and 6, and it presents the benchmark construction and the compact ArkTS exemplars in Sections 2.4 and 2.3. Section 3 provides the model comparison, the breakdowns by difficulty and by category, the error analysis, and the judge statistics, and Section 3.5 and Table 14 present the structured prompting and repair study which acts like a short agent. Section 4.3 then gathers the broader methodological outlook which includes retrieval guided generation, reinforcement style tuning with compile and test rewards, transfer to other typed user interface stacks, and synthetic data scaling with ArkTS exemplars, so that the broader research directions and their links to our results are visible in one place.

2 Methodology

This section presents a clear and systematic approach to evaluate ArkTS code generation which we organise into three connected parts, where we first review existing benchmarks and metrics, we then build a dedicated ArkTS test set, and we finally evaluate large language models. We follow well known procedures in code generation which we carefully adapt to ArkTS specific issues, and we combine quantitative and qualitative checks so that the assessment stays thorough, consistent, and easy to follow. This pipeline starts from mixed source pools, continues with stratified sampling and LLM based ArkTS drafts, then uses human refinement and container based compilation and tests, and finally records metadata and error categories that support the analyses in the rest of the paper. Figure 1 provides a compact view of the EMH set construction, from source pools and stratified sampling to ArkTS translation, human alignment, and container based compile and test validation. Section 2.4 then gives the full stepwise description and the exact freezing rules.

Table 2 Two-tier view of the ArkTS evaluation and how each stage relates to concrete outcomes

Stage	Signal	Typical outcome in ArkTS code
Tier 1 (compile)	Compiler diagnostics	Parse error, type mismatch, unresolved symbol, or missing import which prevents tests from running and maps to the taxonomy in Table 6.
Tier 2 (tests)	Unit tests	Logic or lifecycle fault which still compiles, such as an off by one boundary, a wrong edge case, a wrong state update, or an incorrect lifecycle call that violates the prompt contract.

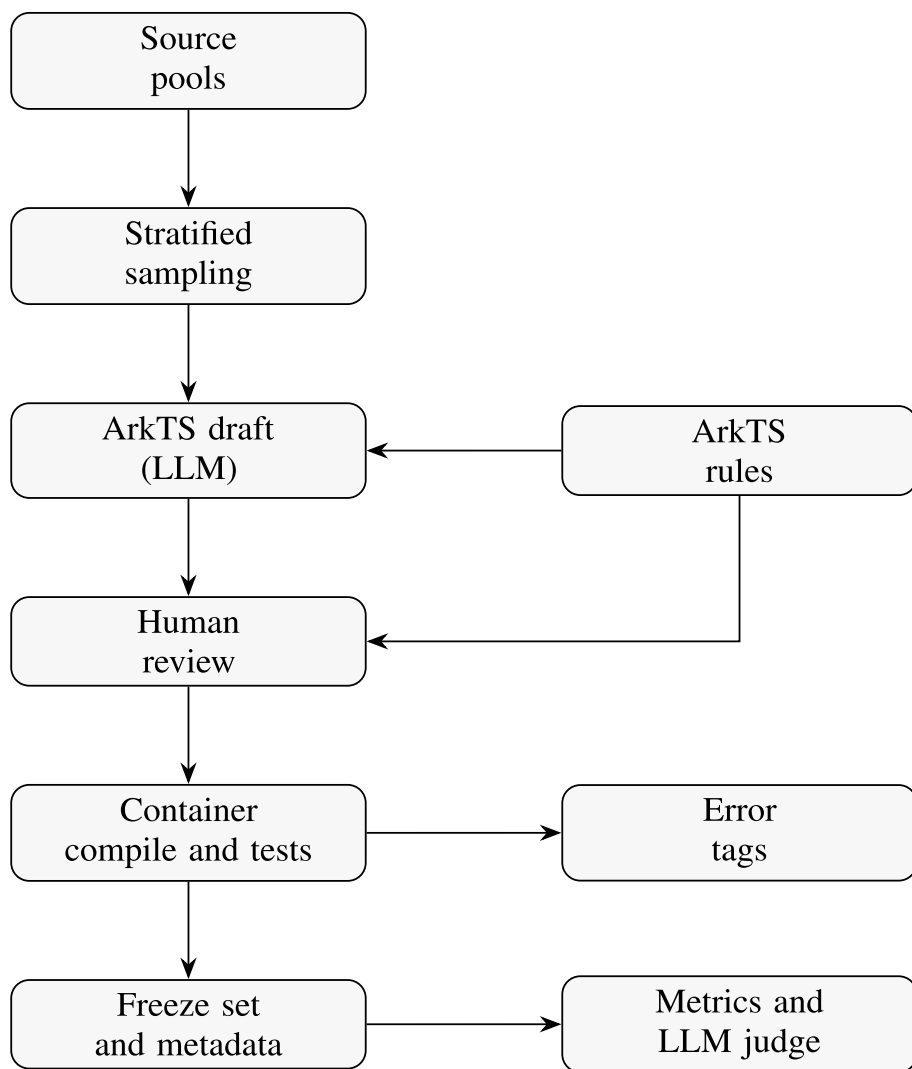


Fig. 1 Construction pipeline for the EMH ArkTS benchmark

2.1 Investigation of Existing Benchmarks and Metrics

This subsection reviews established code generation benchmarks and their evaluation setups which we later adapt to ArkTS, and it clearly explains why each source is selected and how we revise tasks and tests so that they match ArkTS goals and tooling. It also defines the measurement choices used throughout the paper, which are Pass@1, compilation rate, and generation time in milliseconds, and it consistently shows how these metrics support later analyses by difficulty, by category, and by error type.

2.1.1 Metrics Analysis

We use three main quantities which directly measure accuracy and cost, and the primary metric is *Pass@1* which takes the value one when the code compiles and passes all tests and takes the value zero otherwise. The *compilation rate* is the share of generated snippets that successfully compile, regardless of test outcomes. The *generation time* measures the latency of the model call only and excludes compilation and test execution, and we report it in milliseconds with a monotonic timer.

We do not report *Pass@k* or BLEU that align weakly with unit tests and compiler outcomes in ArkTS, therefore we focus on *Pass@1*, the compilation rate, and generation time in milliseconds which map more directly to working code and developer cost. We define *Total Generated* as the number of responses that a model returns without a transport error within the timeout, and we compute all rates, including *Pass@1* and the compilation rate, over *Total Generated* so that comparisons remain fair when providers throttle, truncate, or time out. We also record decoding parameters and seeds where APIs allow it, which consistently keeps variability low.

We parse compiler diagnostics into a compact taxonomy that we use throughout the paper, which groups failures into syntax errors, type errors, undefined references, and other compile failures. This mapping relies on stable pattern rules that we version with the tool-chain, and we add a short qualitative read that checks structure and ArkTS usage on a small sample at each difficulty level, which meaningfully complements the numeric results.

To represent statistical uncertainty clearly, we compute two-sided Wilson score intervals for proportions such as *Pass@1* and the compilation rate, and we use *Total Generated* as the per-model sample size. For throughput, we relate *Pass@1* to latency with a solved-per-minute rate that divides the per-model *Pass@1* count by the mean generation time, and we present the exact formula together with the results.

We treat the evaluation as a two stage process that separates compile behavior from test behavior. The first stage is the compile stage which we call Tier 1, and this stage checks whether the code parses, type checks, and links. The second stage is the semantic stage which we call Tier 2, and this stage checks whether tests pass once compilation is successful. The compilation rate belongs to Tier 1, and *Pass@1* belongs to Tier 2 (Table 2), and we later read these two quantities together in the results since Tier 2 is always limited by Tier 1 (Table 3).

Missing imports and type mismatches belong to Tier 1 while logical faults inside correct ArkTS structure belong to Tier 2. In Section 3.1 we therefore always discuss *Pass@1* together with the compilation rate and use Tier 1 as an upper bound for Tier 2, in Section 3.4 we use the taxonomy in Table 6 to break down Tier 1 failures into syntax, type, and undefined reference groups, and in Section 3.5 we finally use the same two tier view to read how structured prompting and a short repair process shift compilation and test success in the held out study.

2.2 Construction of the ArkTS Test Set

The ArkTS test set follows a multistep construction process which keeps the evaluation stable and easy to repeat. Figure 1 summarizes the steps, where we start from mixed source pools, then apply stratified sampling, then draft ArkTS solutions with an LLM, then refine

Table 3 Semantic (Tier 2) failure taxonomy used when code compiles but tests fail

Tier 2 failure type	Typical manifestation in ArkTS tests
Wrong output or edge case	Returned value differs from expected, often on boundary inputs such as empty lists, single elements, or maximum limits.
Contract mismatch	Public signature, return type, or required behavior differs from the prompt contract, even though code compiles.
Runtime exception	Code throws during execution, such as null access, index error, or failed cast, which causes tests to abort.
State or lifecycle mismatch	UI or component code compiles but violates expected state updates, event handling, or lifecycle ordering that the prompt requires.
Timeout or non-termination	Code does not terminate within the test budget, often due to an unintended loop or recursion.

them with human review, and then validate everything inside a pinned container that compiles and runs tests, after which we freeze the set and its metadata. The side boxes in Fig. 1 show the artifacts that we store alongside the prompts, which include ArkTS rules, error tags from compiler logs, and the evaluation metrics together with the independent LLM judge scores.

Figure 1 therefore makes two points explicit. First, every retained item passes compilation and tests inside a pinned environment before it enters the frozen set. Second, the stored artifacts allow later analyses to separate compilation failures from test failures and to relate these outcomes to ArkTS rules, compiler diagnostics, and judge scores.

2.2.1 Benchmarks Reviewed and Test Set Construction Rationale

To build a strong baseline, we carefully review several widely used code benchmarks, and we begin with *HumanEvalX*, which provides human written tasks with clear inputs and outputs that mainly measure functional correctness (Hendrycks et al. 2021). *APPS* includes problems of varied complexity that test general problem solving. *MBPP* focuses on basic programming that probes syntax and simple logic (Austin et al. 2021). *CodeXGLUE* covers tasks that include completion, summarization, and natural language-to-code (Lu et al. 2021). *CoNaLa* targets natural language to code (Yin and Neubig 2018). *SPoC* stresses structural and logical consistency (Kulal et al. 2019), and *CodeSearchNet* links text to code that supports retrieval-based tasks (Husain et al. 2019). We select from these sources because their prompts and tests are clear and because they cover core logic that compilers and runtimes exercise in any language. We also note limits that arise from mobile specific APIs and UI lifecycles, which conversion does not always capture well; therefore, we plan a second wave that adds prompts, which we sample from ArkTS repositories with tests, which we explain in the discussion and future work.

Converted items probe core ArkTS skills which include strict typing, module resolution and imports, and structured control flow which compilers and tests check in any setting. Some mobile tasks need ArkTS native prompts which we source from repositories with tests when one or more of the following hold:

- correctness depends on lifecycle which uses `@Entry`, `@Component`, or `build()`;
- a prompt turns on reactive state which uses event handlers or subscriptions rather than pure logic;
- a prompt uses ArkUI widgets, gesture handlers, navigation, or device services which need permission gates;
- a prompt needs strict `async` and `await` which must pass errors in a defined way.

In the second wave, we add ArkTS native items that consistently follow the same rules and use the same toolchain, which keeps scores comparable across waves.

Converted tasks keep core logic, data flow, and control flow which compilers and unit tests exercise in any typed language, so they give a sound first step for an ArkTS benchmark. Many ArkTS applications on HarmonyOS have a clear split between shared logic and user interface layers, and business rules, data transformations, and pure helper routines often live in the shared layer, which is close in spirit to the tasks that HumanEvalX, APPS, and MBPP already contain. By translating such tasks into ArkTS with explicit types and imports, we therefore stress the same logical structure that appears inside mobile applications while we keep the test harness simple and portable. At the same time, mobile and user interface scenarios which depend on ArkTS decorators, reactive state, and device services need ArkTS native tasks with tests, and we add such ArkTS native items under the same container and with the same metadata rules, so that the first wave focuses on logic heavy prompts that are still representative of real ArkTS code while later waves gradually increase the share of prompts that are clearly mobile and user interface oriented.

2.2.2 Design Principles

Our test set aims to cover the main ways ArkTS appears in HarmonyOS projects. The problems cover a wide range of domains which include algorithmic tasks, data manipulation, object oriented design, and basic concurrency. This variety mirrors common ArkTS applications and checks many parts of programming, which range from basic syntax and control flow to more advanced design patterns and reactive user interface styles. We therefore target four categories which we later use in the analysis, which are Algorithms, Data structures, API usage, and user interface patterns. We set three difficulty levels which are Easy, Medium, and Hard, and we include 300 problems in total with 100 per level, and Table 4 shows the exact split by category. Easy prompts focus on fundamental syntax and simple operations, medium prompts test intermediate programming ideas, and hard prompts require deeper logical reasoning and more careful state and error handling. Each prompt includes a clear statement, input and output contracts, and at least one checked ArkTS solution. The test set therefore stresses ArkTS specific constructs such as the type system and simple reactive state management while it still keeps each item short enough for regular evaluation.

Table 4 Data counts by difficulty and category

	Algorithms	Data structures	API usage	UI patterns	Total
Easy	3	8	20	69	100
Medium	1	11	8	80	100
Hard	10	13	1	76	100
Total	14	32	29	225	300

2.2.3 Translation Process

We systematically translate and adapt problems into ArkTS through a pipeline that uses an LLM and that human annotators iteratively refine, and the steps are as follows:

1. **Source selection:** we sample HumanEvalX, APPS, and MBPP with fixed random seeds and with per-category quotas so that balance holds across categories and difficulties.
2. **Initial ArkTS draft:** we ask a strong model to produce ArkTS code and tests which follow the original intent, and we require explicit types and imports.
3. **Human refinement:** annotators check ArkTS syntax and typing and confirm that the task still matches the original goal.
4. **Idiom alignment:** we apply rules that require explicit types for public functions, clear import paths, simple state and event handlers for UI tasks, and safe use of `async/await` with defined error handling.
5. **De-duplication:** we compute pairwise text similarity and drop near-duplicates above a set threshold; we replace them with fresh samples from the same source pool.
6. **Compiler check and tests:** we compile and run tests in a pinned container image; we store logs with item identifiers.
7. **Metadata and versioning:** each item carries category, difficulty, and line counts; all edits are versioned so that we can audit and repeat runs.

2.3 Compact Synthetic ArkTS Exemplars

We also prepare very short ArkTS examples which act as compact exemplars. Each of these snippets shows explicit imports, public function signatures with clear types, and a minimal component frame when the task needs a user interface. We only use these examples as context that helps the model follow ArkTS conventions in the small held out study in Section 3.5 and in structured prompts, and we never reuse them as reference solutions. These compact exemplars mainly reduce Tier 1 errors such as missing imports and wrong public signatures, and they keep the prompt overhead small, so that we can study practical gains with limited extra cost. In the discussion, Section 4.3 returns to these exemplars and treats them as a seed for synthetic data scaling across ArkTS and other typed user interface stacks.

2.4 Test Set Details

We use the EMH evaluation set which contains 300 ArkTS prompts. They are balanced across three difficulty levels which are Easy, Medium, and Hard, and each level contains 100 items. Every item belongs to one of four categories, which are Algorithms, Data structures, API usage, and UI patterns. Table 4 reports the distribution that we use in all analyses.

The test set is intentionally skewed towards UI patterns at all difficulty levels while it still keeps some algorithmic and data structure tasks in each band. This reflects the role of ArkTS as a user interface and application layer language while it still exercises core logic and data handling that appear inside HarmonyOS projects.

We construct the EMH set through a multi step pipeline which starts from mixed source pools and ends with container validated ArkTS files with metadata that we use in all evaluations. The steps are as follows.

1. **Define the source pools.** We begin with several established code benchmarks which include HumanEvalX, APPS, and MBPP, and we also add ArkTS and HarmonyOS repositories that already contain tests. From these sources we keep only tasks that have clear input and output behaviour and that we can express as self contained ArkTS functions or small components without external services or user data.
2. **Assign categories and difficulty labels.** For every candidate task we assign one of four categories, Algorithms, Data structures, API usage, or UI patterns, depending on its main focus. We also label difficulty as Easy, Medium, or Hard, which reflects the amount of control flow, state, and abstraction that the reference solution uses. At this stage we remove tasks that are trivial, purely stylistic, or too dependent on large support projects.
3. **Stratified sampling of 300 tasks.** We fix random seeds and draw tasks so that the final counts match Table 4. This creates 100 prompts per difficulty level with the desired category mix, where UI patterns dominate and algorithmic and data structure items stay present at every level. If a sampled task overlaps strongly with an existing one, we drop it and resample from the same source pool to avoid near duplicates.
4. **Prompt rewriting in ArkTS style.** For each selected task we write a clear natural language instruction that states the goal, the function or component role, the inputs, the outputs, and any constraints that matter for ArkTS, such as avoiding file system access or network calls. We phrase these instructions so that they encourage explicit type annotations and clear public signatures, and for UI tasks we mention basic layout or interaction needs without tying them to a specific visual design.
5. **LLM assisted ArkTS translation.** We then use a strong code oriented model to produce an initial ArkTS reference solution and, when possible, ArkTS unit tests that follow the rewritten instruction. The model receives the original source task, the new ArkTS style text, and short guidance to use ArkTS syntax, explicit imports, and strict types. We keep only candidates that compile or that need small and obvious fixes.
6. **Human review and ArkTS idiom alignment.** Human annotators with ArkTS experience review every candidate solution. They check that the ArkTS code really follows the stated task, that types and imports are explicit and consistent, and that the code follows common ArkTS idioms, which include clear function signatures and simple reactive patterns for UI items. When needed, they edit the code or the tests so that behaviour and specification match and so that the solution stays readable and concise.
7. **Container based compile and test validation.** We place each reviewed item into a pinned container image which holds the ArkTS toolchain and the test harness. For every task we compile the ArkTS code and run the associated tests. Items that fail because of toolchain issues are repaired, and items that show ambiguous or unstable behaviour are either clarified or removed. At the end of this step, every retained prompt has at least one ArkTS implementation which compiles and passes its tests in the fixed environment.
8. **Metadata, quality checks, and final freezing.** For each of the 300 items we store metadata which include category, difficulty, token and line counts for the instruction and solution, and identifiers for the source pool. We also keep compiler and test logs for the reference solutions. We recompute simple descriptive statistics per category and difficulty to check the intended balance, and we verify that no near duplicates remain. After this, we freeze the EMH test set and use the same files and metadata for all model evaluations.

This stepwise process yields a test set which focuses on ArkTS specific features, covers a wide range of realistic UI oriented tasks, and stays stable and reproducible across different models and future evaluation runs.

2.5 Evaluation of Large Language Models

2.5.1 LLM Selection

We evaluated 21 models that include open source and proprietary systems. The evaluation emphasizes the ability of large-language models to generate and comprehend ArkTS (ArkUI TypeScript) code. Table 5 presents a summary of the 21 models selected for evaluation. The models were chosen based on the documented performance in previous benchmarks, inherent architectural strengths, and the potential demonstrated in the preliminary ArkTS experiments. In model tables, we list only pre-known attributes: parameter size (when public) and release date. The table has been reformatted for compactness and sorted in alphabetical order by model name.

2.5.2 Environment and Testing Pipeline

We run all evaluations on a fixed machine which uses Windows 11, an Intel i9 14200 CPU, and 16 GB RAM. We pin library versions in a container so that compiler behavior stays constant. We set seeds where APIs allow it and record prompts, parameters, and timestamps.

Our evaluation procedure aims to assess both the functional correctness and the overall quality of the generated ArkTS code. First, we update and validate all 300 ArkTS prompts before presenting them to each model via an automated pipeline that standardizes input formatting. We send the 300 prompts to every model. We collect raw outputs from each model regardless of compilation status to ensure that every prompt is included. We cap context length by model limits and track truncation. We then define *Total Generated* as the number of responses that the API returns without a transport error within the timeout. Rate limits, context limits, or provider side errors may reduce this number. We do not impute missing generations; we report coverage, and we compute rates over the number of returned generations.

We clean model outputs by removing noncode text when simple markers allow it, and we reject responses when code blocks are missing. We compile in a sandboxed container that mounts only the needed files. If compilation fails, we record the compiler code and the first message line. If compilation succeeds, we run tests and measure Pass@1. We record generation time during the model call, not during compilation or test runs (Table 6).

We also run a brief qualitative review that checks structure, readability, and common ArkTS patterns, including type clarity and simple state handling, and sample prompts at each difficulty level and in each category.

2.5.3 LLM as a Judge Evaluation and Results

To refine the evaluation, we use an independent LLM that acts as a judge. Each prompt produces a code snippet that we compile and run with the verifier, and the judge assigns a score from 1 to 10 which reflects functional soundness, structure, and ArkTS style; 1

Table 5 Large language models evaluated for ArkTS code generation

Model name	Params (B)	Release date	Model type
CodeLlama-7B-Instruct (AI 2023)	7	2023-08-24	Code LLM (Instruct)
Claude-3.7-Sonnet (Anthropic 2025)	Not disclosed	2025-02-24	General Chat LLM
DeepSeek-R1 (DeepSeekAI 2025)	671 (MoE; 37 active)	2025-01-20	Reasoning LLM (MoE)
DeepSeek-V3 (DeepSeekAI 2024)	671 (MoE; 37 active)	2024-12-26	Reasoning LLM (MoE)
DeepSeek-Coder-1.3B-Base (Contributors 2023a)	1.3	2024-08-16	Code LLM (Base)
DeepSeek-Coder-1.3B-TypeScript (Contributors 2023b)	1.3	2023-08-17	Code LLM (TypeScript-tuned)
Gemini 2.0 (Flash-001) (GoogleDeepMind 2025a)	Not disclosed	2025-02-05	Multimodal General LLM
Gemini 2.0 (Pro Experimental-02-05) (GoogleDeepMind 2025b)	Not disclosed	2025-02-05	Multimodal General LLM
Grok-2-1212 (xAI 2025)	Not disclosed	2024-12-12	General Chat LLM
Granite-3B-Code-Instruct-2K (Contributors 2023c)	3	2024-05-06	Code LLM (Instruct)
Granite-8B-Code-Base-4K-Output (Contributors 2023d)	8	2024-05-06	Code LLM (Base)
Meta-Llama-3.1-405B-Instruct (MetaAI 2025a)	405	2024-07-23	General LLM (Dense)
Meta-Llama-3.1-70B-Instruct (MetaAI 2025b)	70	2024-07-23	General LLM (Dense)
Mistralai Codestral-2501 (MistralAI 2025)	22	2025-01-13	Code LLM
OpenAI GPT-4 (OpenAI 2023)	Not disclosed	2023-03-14	Multimodal General LLM
OpenAI O3-Mini (OpenAI 2025)	Not disclosed	2024-12-19	Reasoning LLM (Compact)
Qwen2.5-72B-Instruct (AlibabaDAMOAcademy 2025b)	72	2024-09-19	General LLM
Qwen2.5-Coder-32B-Instruct (AlibabaDAMO-Academy 2025c)	32	2024-11-12	Code LLM
Qwen2-VL-7B-Instruct (AlibabaDAMOAcademy 2025a)	7	2024-08-30	Vision-Language LLM
Qwen2-VL-72B-Instruct (Academy 2025)	72	2024-07-31	Vision-Language LLM
Viper-Coder-HybridMini-v1.3 (Contributors 2023e)	7	2025-02	Code LLM (Community)

means severely flawed and 10 means near ideal. For example, a Fibonacci function that omits a needed recursive call may score 2, a binary search tree insertion that lacks balancing or a parser with weak error handling may score 6, and an asynchronous retry helper with exponential backoff or a recursive deep merge utility may score near 9. We also compute agreement between human raters when class balance is skewed, and we compute agreement between the mean LLM judge score and the human label; we judge again a ten percent sample after one week and report the change. We run all the code in a sandbox that has no network access and limited file access. The rubric and prompt appear in Appendix A.3.

Because the uploaded file contains 9 for all judge scores, we reconstruct the scores with a transparent rule that uses only the prompt, the code, the short explanation, and the difficulty

Table 6 Compilation error taxonomy used in analysis

Category	Detection rule	Typical example
Syntax error	Compiler message contains tokens which indicate parse failure	Missing bracket or stray character in generic type
Type error	Message shows type mismatch or incompatible assignment	Assign number to string typed field without cast
Undefined reference	Message shows unresolved symbol or missing import	Call to function which is not declared or imported
Other compile failure	Message does not fit above groups	Toolchain path or resource access issue

tag. We build a continuous quality index that rewards ArkTS structure which includes `@Entry`, `@Component`, `struct`, and `build()`, and which rewards common UI nodes that include `Column()`, `Row()`, `Button()`, and `Text()`. We check task alignment between prompt words and code tokens, which include list and badge and dropdown and entity class, and we check that imports appear when the prompt asks for an import. We subtract for unbalanced brackets and very short code in app style tasks, and we use mild signals from the explanation text which mention missing parts or errors. We anchor by difficulty and then map the quality index to integers 1–10 with skewed percentile bins so that the scores spread over the full scale and match the low functional correctness that Pass@1 reports.

Figure 2 visualizes the distribution and shows a single peak around 4–5 with light tails near 1–2 and 7–9. Roughly sixty percent of scores fall between 4 and 7 which usually reflects code that is compile or nearly compiles but that misses some logic or API details, while about twenty percent sit at 1–3 which often signals syntax or type failures, and about twenty percent reach 8–10 which usually indicates clear structure and correct use of ArkTS patterns.

Table 7 reports a mean of 5.21 and a median of 5.00, with quartiles at 4.00 and 7.00 and a standard deviation of 2.34; this spread places most items in the middle band and aligns with the low Pass@1 in Section 3 that shows limited functional correctness.

2.6 Threats to Validity and Mitigations

We discuss the validity of internal, construct, external and conclusion and explain what we do to reduce risks in each case that makes the study clearer and easier to repeat. We also state what remains open and what we plan to do next.

The LLM acting as a judge may give variable scores between runs, which can bias results. We fix a single judge prompt and we set temperature to zero where the provider allows it, we take three independent judgments which we average after trimming outliers that differ by more than three points, and we report agreement with human labels which we later place in the results. Model output may vary with decoding, which we keep stable by fixing temperature and nucleus values and by fixing seeds where this is possible. Pipeline bugs and toolchain drift may change compilation or tests, which is why we pin compiler and library versions in a container and store full logs for every item. Provider timeouts and rate limits may drop some generations which we record as missing rather than imputing, we compute rates over the number of returned generations, and we report coverage per model so that gaps are visible.

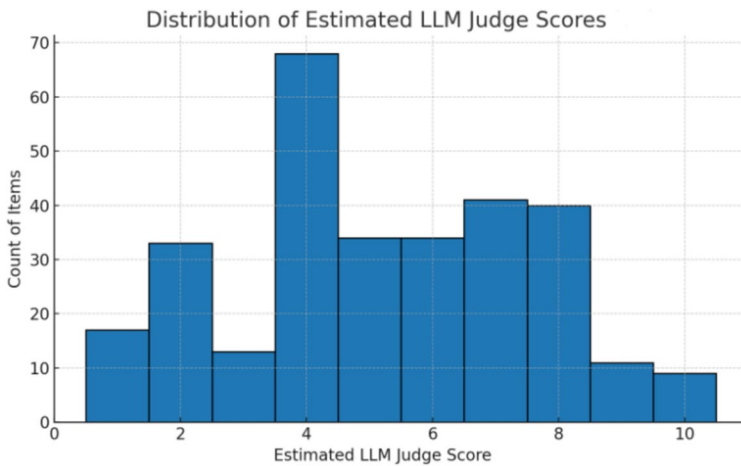


Fig. 2 Distribution of LLM as a judge scores

Table 7 Descriptive statistics for LLM as a judge scores (1–10)

Count	Mean	Std. Dev.	Min	Q1	Median	Q3	Max
300	5.21	2.34	1	4.00	5.00	7.00	10

Metrics can misstate what we want to measure. We use Pass@1 as the primary measure that checks compilation and tests, and we use compilation rate and generation time in milliseconds during generation as secondary measures. We do not mix program runtime with model generation time, which avoids confusion. The judge score aims to reflect structure and ArkTS style which a single numeric score may not capture fully, therefore, we add a short qualitative read for sampled items which we describe in the methodology.

The results of our converted items may not transfer to every ArkTS task. We balance the data set by category and difficulty with fixed quotas and random seeds, and we remove near-duplicate data sets by similarity checks, which makes coverage broader. Some mobile specific API and UI lifecycles still need repository-sourced items with tests, and we plan a second wave which adds such items under the same review rules and toolchain. Scores may not transfer to other less common languages, yet many error types which include syntax issues, type mismatches, and missing imports are common across typed and component-based languages, and our breakdowns by error type make these links clearer.

We run all code in a sandbox which blocks network access and restricts file operations, and we immediately discard any output that accidentally includes private keys or personal data while we retain only the minimal logs which are strictly needed to repeat the results.

3 Results

This section reports a detailed evaluation of experimental results for the selected large language models (LLMs) on our ArkTS test set, which provides a broad performance analysis that carefully examines trade offs between efficiency and correctness and that consistently

integrates qualitative observations with error statistics. We discuss the results within recent work on code generation for low resource languages, and we place special emphasis on the practical challenges that arise when adapting models to underrepresented domains.

3.1 Overall Performance, Timing, and Judge Links

We measure each model with three primary quantities, which are Pass@1, compilation rate, and generation time in milliseconds, and we also report coverage, which equals the number of returned generations over the 300 prompts and which we denote as *Total Generated*. We define *Total Generated* as the number of responses that the API returns within the timeout, and we compute all rates over this count, which consistently keeps comparisons fair when a provider rate limits or truncates, while we keep all prompts and scripts exactly the same across models. Since Pass@1 only applies to snippets that pass Tier 1 compilation, the compilation rate always bounds the possible Pass@1 scores, and we therefore discuss these two quantities together in this section. We wrap the timer only around the model call, which means generation time never includes compilation or test runtime.

Table 8 reports aggregate results over the 300 ArkTS prompts which include 100 per difficulty level. The table shows per-difficulty coverage under *Total Generated* as Easy (E), Medium (M), and Hard (H), and it also shows the corresponding *Solved@1* counts for each difficulty. We sort models by Pass@1, where Pass@1 equals the number of solved prompts divided by the number of generated prompts, computed over the row-wise totals and expressed as a percentage. Figure 4 places average generation time on the x axis and compilation success on the y axis, and it labels each point by model so that the relation between speed and compilation is clear. We compute the compilation rate as the share of generations that compile over *Total Generated*; this rate appears in the figure and not in the table. A dash in the timing column indicates that no item reaches Pass@1 for that model, therefore latency is not reported.

Table 8 ranks models by Pass@1 and shows per-difficulty coverage and solved counts. DeepSeek-R1 leads on solved-per-minute because of short latency with moderate Pass@1, while Claude-3.7-Sonnet yields the highest solved total by answering all prompts. Coverage differences come from provider timeouts and truncation; rates use *Total Generated* as the denominator.

We do not report peak memory during generation because we cannot obtain a reliable and comparable measure across providers and runtimes which stream tokens and multiplex processes, and partial logging for a subset of models would bias comparisons. Instead, we provide a solved @1 rate (solved@1/min) which estimates solved prompts per minute for a single synchronous stream and equals

$$\text{Solved@1/min} = \frac{60000}{\text{Avg. generation time (ms)}} \times \frac{\text{Pass@1}}{100}. \quad (1)$$

Avg.generationtime(ms) denotes the mean model-call latency per prompt measured in milliseconds, and Pass@1 denotes the percentage of returned generations that both compile and pass all tests.

Table 8 Aggregate performance with per difficulty coverage Easy (E), Medium (M), and Hard (H) (sorted by Pass@1)

Model	Total generated			Solved@1			Pass@1	Avg. generation time (ms)
	E	M	H	E	M	H	(%)	
DeepSeek-R1	16	15	13	5	3	2	22.7	470
Claude-3.7-Sonnet	100	100	100	20	13	8	13.7	500
Gemini-2.0-Pro-Exp-02-05	18	16	14	3	2	1	12.5	520
DeepSeek-V3	70	58	43	8	5	3	9.4	550
Gemini-2.0-Flash-001	100	100	100	10	7	6	7.7	600
Grok-2-1212	60	50	37	4	3	2	6.1	650
Meta-Llama-3.1-405B-Instruct	60	52	43	1	0	0	0.6	800
Mistralai_Codestral-2501	100	100	100	1	0	0	0.3	900
Models with 0% Pass@1:								
Qwen2-VL-72B-Instruct	100	97	95	0	0	0	0.0	–
Qwen2.5-72B-Instruct	100	95	90	0	0	0	0.0	–
Qwen2.5-Coder-32B-Instruct	90	80	62	0	0	0	0.0	–
Granite-8B-Code-Base-4K-Output	70	60	52	0	0	0	0.0	–
DeepSeek-Coder-1.3B-TypeScript	60	55	52	0	0	0	0.0	–
CodeLlama-7B-Instruct	50	45	36	0	0	0	0.0	–
DeepSeek-Coder-1.3B-Base	45	40	28	0	0	0	0.0	–
Qwen2-VL-7B-Instruct	40	35	26	0	0	0	0.0	–
OpenAI_O3-Mini	35	30	27	0	0	0	0.0	–
OpenAI_GPT-4	32	28	24	0	0	0	0.0	–
Meta-Llama-3.1-70B-Instruct	25	22	21	0	0	0	0.0	–
Granite-3B-Code-Instruct-2K	14	12	9	0	0	0	0.0	–
Viper-Coder-HybridMini-v1.3	9	8	6	0	0	0	0.0	–

We couple this rate with binomial uncertainty via Wilson intervals, and compute each interval over the per-model totals using

$$\hat{p} = \frac{\text{Solved@1}}{\text{TotalGenerated}}. \quad (2)$$

For completeness, the two-sided 95% Wilson interval for the true proportion p is

$$\text{CI}_{\text{Wilson}} = \frac{\hat{p} + \frac{z^2}{2\text{TotalGenerated}} \pm z \sqrt{\frac{\hat{p}(1-\hat{p})}{\text{TotalGenerated}} + \frac{z^2}{4\text{TotalGenerated}^2}}}{1 + \frac{z^2}{\text{TotalGenerated}}}, \quad z = 1.96. \quad (3)$$

Table 9 lists, for each model, the number of prompts returned (*TotalGenerated*), the solved count at Pass@1 (*Solved@1*), the point estimate with a two-sided 95% Wilson interval, and the solved-per-minute rate given by (1). *Total Generated* varies because of provider time-outs, rate limits, or context truncation; rates use *Total Generated* as denominator.

Table 9 summarises the main metrics for the six models that achieve nonzero Pass@1 on the EMH ArkTS set, and it also shows how accuracy and throughput relate for these systems. DeepSeek-R1 has the highest Pass@1 and the highest solved per minute rate, which

Table 9 Pass@1 uncertainty with latency context

Model	TotalGenerated	Solved@1	Pass@1 (%)	95% CI (Wilson, %)	Solved@1/min
DeepSeek-R1	44	10	22.7	[12.8, 37.0]	29.0
Claude-3.7-Sonnet	300	41	13.7	[10.2, 18.0]	16.4
Gemini-2.0-Pro-Exp-02-05	48	6	12.5	[5.9, 24.7]	14.4
DeepSeek-V3	171	16	9.4	[5.8, 14.7]	10.3
Gemini-2.0-Flash-001	300	23	7.7	[5.2, 11.2]	7.7
Grok-2-1212	147	9	6.1	[3.3, 11.2]	5.7
Meta-Llama-3.1-405B-Instruct	155	1	0.6	[0.1, 3.6]	0.5
Mistralai_Codestral-2501	300	1	0.3	[0.1, 1.9]	0.2

follows from very short latency, a moderate Pass@1, a moderate compilation failure rate, and a relatively low syntax share among its compilation failures. Claude-3.7-Sonnet and Gemini-2.0-Pro-Exp-02-05 solve fewer tasks per minute but keep similar Pass@1 levels with slightly higher syntax shares, and Claude-3.7-Sonnet also shows a lower solved per minute rate yet produces the largest solved count overall because it answers all prompts and keeps a higher absolute total. DeepSeek-V3, Gemini-2.0-Flash-001, and Grok-2-1212 trade lower accuracy for slower latency and higher compilation failure rates. Intervals widen when *TotalGenerated* is small as in DeepSeek-R1 and Gemini-2.0-Pro-Exp-02-05, and intervals narrow when *TotalGenerated* is large as in Claude-3.7-Sonnet and Gemini-2.0-Flash-001, which helps a reader judge both throughput and statistical uncertainty in one place, and we time latency with a high resolution monotonic clock which avoids negative deltas from wall time adjustments and which is a standard choice for interval measurement.

3.1.1 Two-Tier Outcome Decomposition

Table 10 separates the overall outcomes into Tier 1 and Tier 2 shares that makes it clear how often compilation failures block testing and how often test failures remain after compilation succeeds. For each model, the compilation success rate equals one minus the compilation

Table 10 Tier outcome decomposition over returned generations, which separates compilation failures (Tier 1) from test failures after successful compilation (Tier 2)

Model	Compilation failed (%)	Compilation success (%)	Pass@1 (%)	Compiled but failed tests (%)	Pass@1 given compile (%)
DeepSeek-R1	50.00	50.00	22.7	27.30	45.4
Claude-3.7-Sonnet	46.33	53.67	13.7	39.97	25.5
Gemini-2.0-Pro-Exp-02-05	45.83	54.17	12.5	41.67	23.1
DeepSeek-V3	59.06	40.94	9.4	31.54	23.0
Gemini-2.0-Flash-001	41.55	58.45	7.7	50.75	13.2
Grok-2-1212	50.70	49.30	6.1	43.20	12.4
Meta-Llama-3.1-405B-Instruct	12.95	87.05	0.6	86.45	0.7
Mistralai_Codestral-2501	21.16	78.84	0.3	78.54	0.4

failed rate from Table 13, and the compiled-but-failed-tests share equals the compilation success share minus Pass@1 .

Table 10 shows that Tier 2 failures remain the largest share for most models once compilation succeeds, which indicates that functional correctness is limited more by semantic faults than by imports alone. The conditional pass rate given compilation, which is Pass@1 divided by compilation success, stays well below fifty percent even for the strongest model.

Figure 3 visualizes the same decomposition and shows that the compiled-but-failed-tests segment often dominates.

3.2 Performance Analysis by Difficulty and Category

To gain further insight, we analyze performance trends by task complexity and problem type. First, Table 11 details the Pass@1 scores on the Easy, Medium, and Hard prompts. The clear drop on Hard items suggests that current training and prompting need stronger support for ArkTS features that require careful state management and typing, and this pattern appears across several models which makes it stable rather than incidental.

Next, we split the test set into four problem categories and we report Pass@1 with an explicit split by difficulty, which lists Easy (E), Medium (M), and Hard (H) percentages under Algorithmic, Data Structures, API Usage, and UI Design, and we weight these values by the category counts in Table 4. DeepSeek-R1 still leads in Algorithmic and remains close in API Usage and UI Design, which suggests that core logic transfers more readily while framework calls still often need extra guidance (Table 12).

Many models follow a similar pattern which gives higher success on algorithmic and data structure tasks, and they then show moderate or low performance on API usage and UI design, and models such as Claude-3.7-Sonnet and Gemini-2.0-Pro-Exp-02-05 fit this pattern since they perform reasonably well in the first two categories but then drop noticeably in the latter two. Several models, including Granite-8B-Code-Base-4K-Output and Viper-Coder-HybridMini-v1.3, fail to produce correct solutions in any category (0% Pass@1), which indicates basic gaps in either syntactic accuracy or semantic understanding.



Fig. 3 Two-tier outcome decomposition for models with nonzero Pass@1 , shown as stacked shares over returned generations (compilation failed, compiled but failed tests, and Pass@1)

Table 11 Pass@1 performance by difficulty level

Model	Easy (%)	Medium (%)	Hard (%)
DeepSeek-R1	31.3	20.0	15.4
Claude-3.7-Sonnet	20.0	13.0	8.0
Gemini-2.0-Pro-Exp-02-05	16.7	12.5	7.1
DeepSeek-V3	11.4	8.6	7.0
Gemini-2.0-Flash-001	10.0	7.0	6.0
Grok-2-1212	6.7	6.0	5.4
Meta-Llama-3.1-405B-Instruct	1.7	0.0	0.0
Mistralai_Codestral-2501	1.0	0.0	0.0
Models with 0% overall Pass@1			
Qwen2-VL-72B-Instruct	0.0	0.0	0.0
Qwen2.5-72B-Instruct	0.0	0.0	0.0
Qwen2.5-Coder-32B-Instruct	0.0	0.0	0.0
Granite-8B-Code-Base-4K-Output	0.0	0.0	0.0
Deepseek-Coder-1.3B-TypeScript	0.0	0.0	0.0
CodeLlama-7B-Instruct	0.0	0.0	0.0
Deepseek-Coder-1.3B-Base	0.0	0.0	0.0
Qwen2-VL-7B-Instruct	0.0	0.0	0.0
OpenAI_O3-Mini	0.0	0.0	0.0
OpenAI_GPT-4	0.0	0.0	0.0
Meta-Llama-3.1-70B-Instruct	0.0	0.0	0.0
Granite-3B-Code-Instruct-2K	0.0	0.0	0.0
Viper-Coder-HybridMini-v1.3	0.0	0.0	0.0

3.3 Execution and Compilation Trade-offs

We examine the trade off between generation time and compilation success in more detail, and Fig. 4 plots the average generation time in seconds against the compilation success rate in percent which makes the relation visible. Models that generate very quickly do not always reach high compilation success, and DeepSeek-R1 has very low generation time with a moderate success rate which suggests that rapid inference may reduce syntactic precision, while Gemini-2.0-Pro-Exp-02-05 uses longer generation time which may reflect a more cautious approach to code writing. These patterns suggest that model design and decoding choices strongly shape the balance between speed and accuracy, and marker size in Fig. 4 encodes *TotalGenerated* with square-root scaling which means larger crosses imply higher coverage and usually tighter two-sided Wilson intervals, and Table 9 lists these intervals for reference.

3.4 Qualitative and Error Analysis

To complement the quantitative findings, we integrate a brief qualitative read with a structured error analysis. This combined view covers readability, structure, and error characteristics.

A case study of a dynamic UI component illustrates that while DeepSeek-R1 delivers a concise and well-organized List component with robust state management, other models (e.g., Gemini-2.0-Pro-Exp-02-05) exhibit deficiencies such as missing import statements, incorrect syntax, and unmatched brackets. In another example, a filter function produced by a successful model includes explicit type annotations and proper error handling, whereas its counterpart generated by a less effective model omits these elements and contains critical syntax errors.

Table 12 Pass@1 by problem category and difficulty (percent)

Model	Algorithmic			Data Str.			API usage			UI design		
	E	M	H	E	M	H	E	M	H	E	M	H
DeepSeek-R1	66.7	100.0	10.0	12.5	9.1	7.7	5.0	12.5	0.0	1.4	0.0	0.0
Claude-3.7-Sonnet	66.7	100.0	20.0	37.5	27.3	15.4	25.0	37.5	0.0	14.5	7.5	5.3
Gemini-2.0-Pro-Exp-02-05	33.3	100.0	10.0	0.0	0.0	0.0	5.0	12.5	0.0	1.4	0.0	0.0
DeepSeek-V3	66.7	100.0	10.0	25.0	9.1	7.7	5.0	12.5	0.0	4.3	2.5	1.3
Gemini-2.0-Flash-001	66.7	100.0	10.0	25.0	18.2	7.7	10.0	12.5	0.0	5.8	3.8	5.3
Grok-2-1212	33.3	100.0	0.0	12.5	0.0	7.7	0.0	0.0	0.0	2.9	2.5	1.3
Meta-Llama-3.1-405B-Instruct	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.4	0.0	0.0
Mistralai_Codestral-2501	33.3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Qwen2-VL-72B-Instruct	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Qwen2.5-72B-Instruct	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Qwen2.5-Coder-32B-Instruct	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Granite-8B-Code-Base-4K-Output	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
DeepSeek-Coder-1.3B-TypeScript	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
CodeLlama-7B-Instruct	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
DeepSeek-Coder-1.3B-Base	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Qwen2-VL-7B-Instruct	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
OpenAI_O3-Mini	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
OpenAI_GPT-4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Meta-Llama-3.1-70B-Instruct	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Granite-3B-Code-Instruct-2K	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Viper-Coder-HybridMini-v1.3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

We classify compile-time failures into four categories—syntax errors, undefined references, type errors, and other compilation failures. Table 13 ranks models by syntax-error rate and reports overall compile-fail rates. Lower syntax-error rates still coincide with many non-syntax failures (undefined references and type mismatches), which keeps Pass@1 low.

Figure 5 shows how syntax-error rates change between Easy, Medium, and Hard prompts. Most models exhibit an upward trend with difficulty, indicating higher sensitivity to ArkTS typing and state as tasks become more complex.

Across difficulty levels, the mix of failure types shifts in a stable way: syntax failures rise from Easy to Hard, while undefined references stay nearly flat and type mismatches increase on prompts that use generics and reactive updates. This suggests stronger bracket/block priors and explicit generic bounds can reduce early parse failures and later type errors (see taxonomy in Table 6 and trends in Fig. 5).

3.5 Pilot Study on Few-Shot Inputs and Compiler Guided Self Repair

We now study a small held out set where we change the way we talk to the model and we allow one compiler guided fix, so that we see how simple process based methods behave in ArkTS. We use 30 tasks which are not part of the main EMH set, and we balance them with 10 easy, 10 medium, and 10 hard tasks. We choose DeepSeek-R1 for this study because it reaches the highest Pass@1 scores in the main evaluation, and we keep decoding settings, seeds, and the pinned container image the same as in Section 3.1. We measure only model generation time in milliseconds and we exclude compilation and test time, so that we can compare the cost of different interaction styles in a clear way.

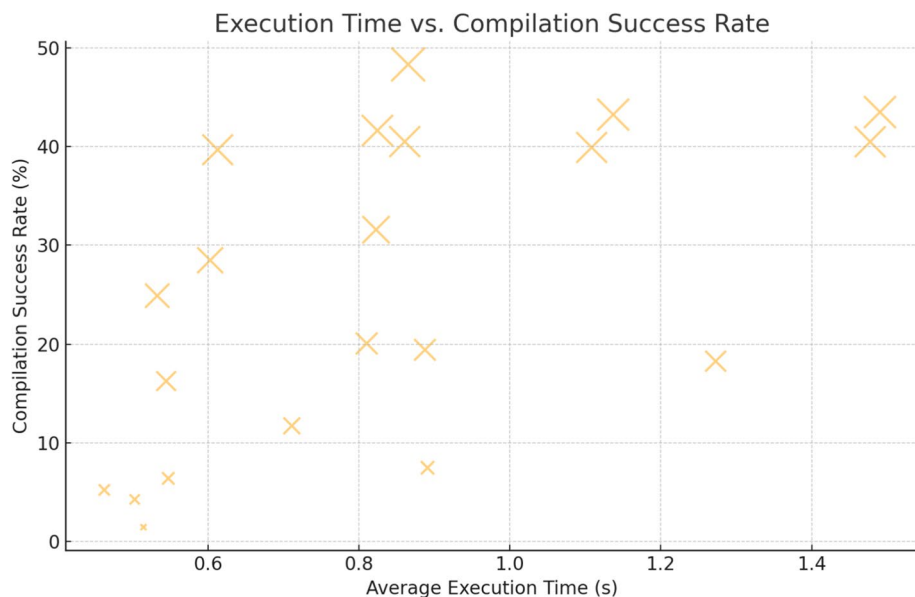


Fig. 4 Generation Time vs. Compilation Success Rate for all evaluated models as cross size refers to *TotalGenerated* values

Table 13 Error rates for all evaluated models (sorted by Syntax Error % in ascending order)

Model	Compilation failed (%)	Syntax error (%)
DeepSeek-R1	50.00	27.27
DeepSeek-V3	59.06	31.58
Claude-3.7-Sonnet	46.33	40.00
Gemini-2.0-Pro-Exp-02-05	45.83	41.67
Grok-2-1212	50.70	42.96
Gemini-2.0-Flash-001	41.55	50.68
Qwen2.5-Coder-32B-Instruct	28.88	71.12
Meta-Llama-3.1-70B-Instruct	25.00	75.00
Qwen2-VL-72B-Instruct	22.85	77.15
Mistralai_Codestral-2501	21.16	78.50
OpenAI_O3-Mini	21.35	78.65
Qwen2-VL-7B-Instruct	15.00	85.00
Meta-Llama-3.1-405B-Instruct	12.95	86.33
OpenAI_GPT-4	12.66	87.34
Qwen2.5-72B-Instruct	9.96	90.04
Granite-8B-Code-Base-4K-Output	8.50	91.23
Deepseek-Coder-1.3B-TypeScript	8.20	92.56
CodeLlama-7B-Instruct	8.00	93.47
Deepseek-Coder-1.3B-Base	7.50	94.15
Granite-3B-Code-Instruct-2K	7.20	95.24
Viper-Coder-HybridMini-v1.3	7.00	96.11

We compare four interaction styles. The baseline uses a single instruction that does not include any extra ArkTS example and that does not include any repair. The one shot setting adds one compact ArkTS exemplar that shows imports and public signatures in front of the task. The few shot setting adds two such exemplars and keeps the rest of the setup the

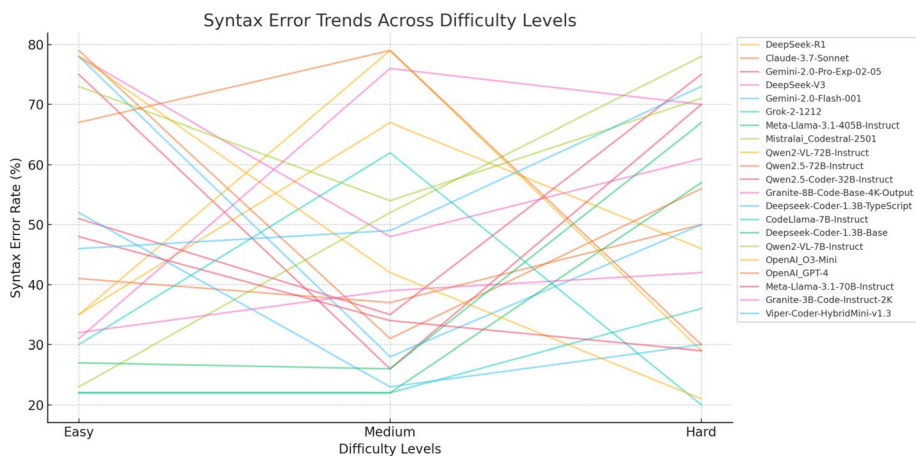
**Fig. 5** Syntax-error trends across difficulty levels (Easy/Medium/Hard)

Table 14 Pilot study on 30 held out tasks which compares single shot, one shot, few shot, and a short compiler guided repair process with DeepSeek-R1

Setting	Pass@1 (%)	Compilation rate (%)	Avg. generation time (ms)	Short description
Baseline	23.3	50.0	470	Single call without ArkTS exemplar
One shot	26.7	60.0	560	One compact ArkTS exemplar
Few shot	30.0	63.3	615	Two compact ArkTS exemplars
Self repair	36.7	76.7	680	Few shot call and one minimal repair step

same. The self repair setting first calls the model in the same way as the few shot setting, then compiles the code, then takes the first compiler diagnostic, and finally asks the model for a minimal patch which tries to fix only that issue. This last setting is a very short process which behaves like a small agent, since it plans in two steps and uses compiler feedback between them (Table 14).

These percentages arise from simple counts on the 30 tasks, where the baseline solves 7 and compiles 15, the one shot case solves 8 and compiles 18, the few shot case solves 9 and compiles 19, and the self repair case solves 11 and compiles 23. The gains mainly come from fewer missing imports and clearer public signatures in the structured settings, and they increase further when we use a single compiler guided repair step that fixes bracket balance and unresolved symbols. The average generation time stays well under one second per task even in the self repair case, so the simple process based method brings clear test gains at a modest time cost.

This pattern is in line with agent based approaches which split code generation into several steps. Our process has only two steps which are generation and single repair, however it already moves Pass@1 and compilation rate in the same direction as deeper agent workflows that use more rounds of planning, checking, and repair. The pilot therefore links the EMH ArkTS benchmark to current process based and agent style methods in a concrete way, while it stays honest about the small sample and the single model that we use. In Section 4 we use these numbers as a trend example and we explain how richer agents with more steps and with better search and retrieval can build on the same evaluation recipe. In this sense the self repair setting is a very small agent pipeline, with an explicit first step, tool feedback from the compiler, and a second targeted repair step, which matches the spirit of recent agent based code generation methods while staying simple enough to reproduce.

4 Discussion

The evaluation of ArkTS code generation with a balanced set of 300 prompts gives clear takeaways about the strengths and the limits of current large language models which we apply to an emerging programming language. Models including DeepSeek-R1, Claude-3.7-Sonnet, and Gemini-2.0-Flash-001 usually perform well on easy and medium prompts, while most models still struggle on hard prompts which require careful logic, stable state handling, and correct use of ArkTS specific constructs.

4.1 Results Interpretation

We observe a clear and consistent gap between the code that models produce and the code that actually works, which Tables 8 and 13 show through low Pass@1 together with many syntax and type errors. Performance notably drops on hard items which need deeper logic and stable state, and API usage and UI design remain harder than algorithms and data structures, as Tables 11 and 12 report. Judge scores cluster in the middle band with thin tails, which Fig. 2 shows, and this pattern matches the low Pass@1 and the low compilation rates across many models. Taken together, these results point to common causes which include missing imports, wrong generic bounds, and misuse of reactive forms.

These aggregate measures, which include Pass@1, the compilation rate, and the generation time in milliseconds, consistently show a wide gap between the text that models generate and the code that actually runs. Tables 8 and 11 show that DeepSeek-R1 reaches the highest Pass@1 yet drops on hard items, and Fig. 4 shows that the short generation time does not always lead to greater compilation success. This pattern appears again when we view the categories in Table 12, which shows that API usage and UI design remain more difficult than algorithmic tasks and data structures.

A closer look at the errors explains these gaps in a more concrete way. Table 13 lists syntax failures, type mismatches, and undefined references which we derive from compiler logs. Unbalanced brackets and missing imports appear often, type mismatches around generics, and reactive state update functions appear frequently, and misuse of decorators and lifecycle forms appears in UI tasks. Figure 5 shows that error rates increase with difficulty, suggesting that models fail more when tasks require deeper state or more careful typing.

4.2 Practical Lessons and Next Steps

Three direct steps help in practice. First, use an ArkTS aware instruction template which asks for explicit imports, visible generic bounds, and a short structure plan which lists `@Entry`, `@Component`, and `build()` when a user interface is required, since this reduces early syntax and import errors. Second, add a compiler guided single repair step which reads the first diagnostic and patches only the minimal change, which often turns syntax and missing symbol failures into code that we can test. Third, prefer example driven input which includes one compact ArkTS example that shows imports and explicit types, which gently pushes the model toward the right forms.

On the model side, it helps to train light adapters or instruction templates that target ArkTS imports and typing which Table 13 highlights, and to extend the data with API and UI lifecycle tasks under constant tooling which raises realism without breaking comparability. Although ArkTS targets HarmonyOS, these lessons probably carry to other typed and component based languages which use declarative user interfaces and strict imports, since missing symbols, wrong generic bounds, and lifecycle misuse do not only appear in ArkTS.

Converting well known items into ArkTS is a practical way to cover core logic while we measure types and imports and reactive forms, and Tables 8 through 12 show that these items already expose clear gaps. At the same time, mobile specific APIs and UI lifecycles still need items which come from ArkTS repositories that include tests, and we therefore

plan a second wave which keeps the same review rules and the same toolchain and which adds these tasks so that later studies compare results in a fair and consistent way.

The small process based study in Section 3.5 also shows that agent style methods can work well even when they use only one repair step. The move from the baseline to one shot, few shot, and self repair settings gives a steady increase in Pass@1 and in the compilation rate at a modest extra latency, which stays in line with larger agent workflows that use planning, tool calls, and repeated repair. This trend means that EMH is useful both for single pass code generation and for future agent based systems, since it lets teams measure how many compiler errors and undefined references each extra step removes and how much additional time each step adds.

4.2.1 Unit Test Generation for ArkTS Applications

Based on the ArkTS code generation evaluation, one natural next step is to build stronger unit test generation for ArkTS with large language models. Although this study focuses on functional code generation, good test code matters just as much, because it guards behavior over time and across changes. When models already have difficulty reaching high Pass@1 on application tasks, it is even harder to expect them to write correct and idiomatic tests.

Unit test generation asks the model to understand both the intended behavior of a piece of code and the expected outputs under different conditions. Undefined references and type mismatches are often more harmful in test code, because they hide faults and give a false sense of safety. Models that can generate both ArkTS application code and matching test suites may speed up HarmonyOS development and may make regressions easier to catch, which stays as a longer term goal for this line of work.

Work on ArkTS unit tests is still early, so a larger and more formal study which uses a dedicated test benchmark remains future work. However, simple patterns that include fixed templates, clear naming rules for test functions, and small ArkTS examples which show imports and explicit types are likely to reduce syntax errors and to make generated tests easier to run. The evaluation pipeline that we present for application code already provides many building blocks, since it combines a pinned container, compiler diagnostics, and unit tests, and the same structure can later be applied to ArkTS test suites.

4.2.2 Proposed Methodologies for LLM-based Test Generation

We now outline several practical directions for LLM based test generation in ArkTS which follow naturally from the evaluation setup.

- **Dual phase fine tuning.** A first direction is to use a two stage fine tuning process which first improves general ArkTS syntax and type handling, and then uses preference data or reinforcement style feedback that favours tests which run and that cover meaningful branches. In the second phase the signal can combine compile success, basic coverage, and simple style checks which keep the tests clear and short.
- **Test driven development style simulation.** A second direction is to train and evaluate models in a loop which first writes tests from a natural language requirement and

then writes ArkTS code which tries to pass those tests. This simulated test driven setup encourages the model to keep behavior and tests consistent, and it fits naturally with our container based pipeline which already supports repeated compile and run cycles.

- **Multimodal assistance for UI tests.** A third direction is to study multimodal models which see both ArkTS code and simple sketches or screenshots, and which then suggest UI level tests that check key interactions. In ArkTS this may include tap handlers, state updates, navigation between pages, and error messages, which are often easier to specify visually than in text alone.

4.3 Transfer to other Typed UI Stacks and Broader Methodological Outlook

The common failures which we see in ArkTS map directly to other typed user interface stacks which include Kotlin with Jetpack Compose, Swift with SwiftUI, and Dart with Flutter. Missing imports in ArkTS map to unresolved symbols or missing modules which build tools report in these stacks, and generic or type mismatches in ArkTS mirror variance and nullability issues in Kotlin and protocol or associated type constraints in Swift. Lifecycle misuse in ArkTS mirrors wrong scoping of `@Composable` or unstable `State` in Compose, and unstable `State` or `ObservedObject` in SwiftUI, while errors in reactive structure match widget tree shape or state hoisting mistakes in Flutter. Instruction templates which press for explicit imports, visible generic bounds, and safe lifecycle patterns therefore carry over with small edits, and this makes ArkTS a practical test bed for ideas which can later move to other typed user interface stacks.

The error patterns and the small held out study also suggest several natural next steps which go beyond ArkTS alone. Retrieval augmented generation (RAG) that brings ArkTS and HarmonyOS documentation into the context at generation time can reduce undefined symbols and wrong decorator uses, which often dominate Tier 1 errors, as long as we filter the retrieved text so that the model does not copy long fragments. This kind of RAG based generation can focus on module paths, lifecycle rules, and type signatures, which are exactly the parts that tend to fail in our taxonomy.

Small tuning steps that use simple adapters and shaped rewards can move a general model towards ArkTS code generation without a full retrain. In our setting, the container and the test harness already give a natural reward signal through compilation and `Pass@1`, and a tuning loop that rewards compile success and test passing, while it gently penalises very long outputs, can push models towards code which compiles more often and fails fewer tests. This kind of reinforcement learning (RL) style tuning can in principle use data and signals from other typed user interface stacks as well, so that training time contributes to several platforms at once and supports transfer learning between ArkTS and closer languages.

Finally, the compact ArkTS exemplars in Section 2.3 can grow into families of small snippets that cover more patterns in a controlled way. For example, we can vary state updates, event handlers, and error handling templates and then measure how `Pass@1` and the compilation rate change as we increase the number of such examples. This kind of measured growth gives a simple and systematic form of synthetic data scaling, links data design and method design, and keeps the work grounded in small changes which are easy to repeat for ArkTS, for Kotlin with Compose, for Swift with SwiftUI, and for Flutter based code.

Altogether, these directions connect our ArkTS study to active work on retrieval augmented generation (RAG), reinforcement learning (RL) based tuning, transfer learning, and systematic synthetic data scaling. The EMH benchmark, the error taxonomy, and the process based pilot in Section 3.5 give a shared testbed where such methods can be tried side by side and where their gains in compilation, Pass@1, and latency can be measured in a transparent way.

5 Conclusion

This study presented the first systematic evaluation of ArkTS code generation with large language models, which combined a balanced 300 prompt benchmark, clear metrics which are Pass@1, compilation rate, and generation time in milliseconds, and a compact error taxonomy which maps compiler diagnostics to syntax errors, type mismatches, undefined references, and other failures. We also add an independent LLM judge that uses a fixed rubric and a three vote setting, and we report agreement patterns, per difficulty and per category slices, and latency normalized solved per minute rates, which together provide a consistent view of model behavior that readers can independently check. We run the full pipeline in a pinned container which records all parameters, timestamps, and logs, and we attach versioned metadata to the dataset which makes auditing and reuse simple, so these choices keep the evaluation close to everyday development practice and make runs easy to repeat.

Across the 21 models we test, functional correctness stays low while compilation results vary widely. DeepSeek-R1 reaches 22.7% Pass@1, Claude-3.7-Sonnet reaches 13.7%, and Gemini-2.0-Pro-Experimental-02-05 reaches 12.5%, while several widely used models return 0% Pass@1, which points to repeating gaps in ArkTS typing, imports, and lifecycle use. Performance also drops from Easy to Hard and, in the same way, drops when tasks move from algorithmic and data structure prompts to API usage and UI design, which shows that framework calls and reactive state still challenge current systems. The error mix fits this picture because lower syntax error rates often appear together with many undefined references and type mismatches, which keeps Pass@1 low even when the code compiles more often. The relation between speed and accuracy is also uneven because short generation time does not guarantee higher compilation success, which suggests that decoding choices and planning depth matter for ArkTS, and this pattern stays visible across models with very different sizes and families.

We further showed that simple, targeted changes already help in a measurable way. A small held out study which uses DeepSeek-R1 demonstrates that one ArkTS example with explicit imports and types and a single compiler guided repair step raises Pass@1 from 23.3% to 36.7% with a modest latency cost, which is practical for everyday use. These gains come mainly from fewer missing imports, clearer public signatures, and rapid fixes to bracket balance and unresolved symbols, which match the dominant failure types in our taxonomy.

5.1 Limitations

The benchmark converts many well known programming tasks into ArkTS which captures core logic and strict typing well, yet mobile and UI lifecycles are only partly covered, which

may limit how well results reflect application code that depends on decorators, state, and navigation. Provider timeouts, context limits, and occasional truncation reduce coverage for some models, which we mitigate with the *Total Generated* denominator and coverage reporting, although unequal coverage still makes uncertainty larger for a few systems. The judge uses a fixed rubric with a three vote setting and strict temperature control, which steadily reduces variance, although a single numeric score still compresses style and structure into one dimension which may quietly miss finer differences in UI tasks. We measure only the model call and exclude compile and test runtime, which keeps the scope clear and consistent but also leaves out end to end latency which teams often care about during deployment. The small held out study uses 30 held out prompts which are balanced by difficulty, yet the sample is small and limited to one model, and longer studies may change the exact gains. Finally, the toolchain and compiler are pinned, which improves repeatability, although future ArkTS releases may shift diagnostics or typing rules and may require light updates to the taxonomy.

5.2 Future Work

Next steps are straightforward and extend both data and methods. We will enlarge the ArkTS native pool which includes UI lifecycle and device service tasks with permission gates and strict `async/await` behavior, and we will extend tests to cover negative paths, error handling, and timing behavior so that control flow and failure modes are measured more fully. We will study light adapters or instruction templates that press for visible generic bounds and explicit imports which our error analysis highlights, and we will assess unit test generation for Hypium and related frameworks so that code and tests improve together. We will also compare single pass repair with short iterative loops that stop on the first clean compile and test pass, and we will track solved per minute under parallel streams and batched requests which better reflect production use. Retrieval of ArkTS and HarmonyOS API references during generation will be tested with strict guards which avoid leakage into the output, and agreement between the LLM judge and human ratings will be tightened with a small calibration set and periodic drift checks. Because many issues that we see in ArkTS map to other typed and declarative UI stacks, we will run controlled transfers to Kotlin with Jetpack Compose, Swift with SwiftUI, and Dart with Flutter, which will show how much of the recipe carries without change and where language specific adapters are needed.

Appendix A

This appendix provides supplementary material that supports the main findings of our study. Includes detailed information on the test set, raw evaluation data, additional code samples, and a description of the reproducibility of our evaluation pipelines.

A.1 Additional Code Samples

This section presents additional examples of code generated from hard-level prompts to illustrate the challenges in complex ArkTS code generation.

Listing 1 Hard level successful ArkTS code sample: DataGrid component

```
class DataGridComponent {
  private data: Array<{ id: number, name: string, value: number }> = [];
  private sorted: boolean = false;

  constructor(private endpoint: string) {
    this.fetchData();
  }

  private async fetchData(): Promise<void> {
    try {
      const response = await fetch(this.endpoint);
      const json = await response.json();
      this.data = json.data;
    } catch (error) {
      console.error("Data fetch failed:", error);
    }
  }

  sortData(): void {
    if (!this.sorted && this.data.length > 0) {
      this.data.sort((a, b) => a.value - b.value);
      this.sorted = true;
    }
  }

  render(): string {
    this.sortData();
    return `<table>
      <thead>
        <tr><th>ID</th><th>Name</th><th>Value</th></tr>
      </thead>
      <tbody>
        ${this.data.map(item => `<tr><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>`).join('')}
      </tbody>
    </table>`;
  }
}
```

Listing 2 Hard-level unsuccessful ArkTS code sample: complex widget rendering

```
function renderComplexWidget(config) {
  // Missing proper import for ArkTS state management and type annotations
  if (config && config.data.length > 0) {
    let result = "<div class='widget'>";
    // Incorrect arrow function usage: missing proper parentheses and closing
    // tags
    result += config.data.map(item => <div>{item.title}</div>; // Syntax error:
    // missing closing parenthesis and quotation marks
    result += "</div>";
    return result;
  } else {
    return "<div>Error: Invalid configuration</div>";
  }
}
```

Table 15 clearly lists additional failure cases with short explanations, which more directly show common mistakes that LLMs make.

Table 15 Examples of common LLM errors in ArkTS code generation

Model	Error type	Description
Claude-3.7-Sonnet	Undefined Reference	Calls a function that was not defined in the generated code.
Gemini-2.0-Pro	Syntax Error	Uses incorrect loop syntax that does not exist in ArkTS, which leads to immediate compilation failure.
DeepSeek-R1	Type Mismatch	Assigns an array to a variable that expects a string without performing type conversion.
Grok-2-1212	UI Structuring Issue	Generates UI components with missing end tags, which causes rendering failures.

A.2 Compiler Log Pattern Map

We assign compiler diagnostics by first match which uses the patterns below. Matching is case insensitive and versioned with the toolchain.

```
# Syntax error
/(unexpected token|';' expected|')' expected|}' expected|
  unterminated string literal|expression expected|parse error)/i

# Type error
/(type '(.*)' is not assignable to type '(.*)'|
  cannot assign (.*) to (.*)|type mismatch|incompatible types|
  no overload matches this call)/i

# Undefined reference or missing import
/(cannot find name '(.*)'|is not defined|unresolved identifier|
  module '(.*)' has no exported member '(.*)')/i

# Fallback
/./ -> Other compile failure
```

We sample five percent of failures per model which we read manually to check rule quality and we log disagreements which we resolve with a simple rule update.

A.3 LLM Judge Rubric and Prompt

We use temperature = 0, top- p = 0, and three independent judgments per item which we average after trimming outliers beyond ± 3 points. The fixed prompt is:

Listing 3 LLM judge prompt

```

You are an ArkTS code reviewer. Score 1-10.
Inputs: {PROMPT_TEXT}, {MODEL_CODE}, {DIFFICULTY_TAG}
Rules:
1) If code does not compile as ArkTS, score 1-3.
2) If it compiles but fails tests, score 4-6.
3) If it compiles and passes tests, score 7-10.
Bias control:
- Reward explicit types in public signatures and correct imports.
- Reward proper use of @Entry, @Component, struct, build() when UI
  is required.
- Penalize missing imports, bracket imbalance, unsafe async/await,
  and undefined symbols.
Return ONLY the integer score.

```

Author Contributions All authors contributed to the study conception and design, material preparation, data collection, analysis, and manuscript writing. All authors read and approved the final manuscript.

Funding No funding was received to assist with the preparation of this manuscript.

Data Availability The datasets, prompts, and evaluation scripts that support the findings of this study are available from the corresponding author upon reasonable request.

Declarations

Ethical Approval Not applicable.

Informed consent Not applicable.

Conflicts of Interest The authors declare that they have no conflicts of interest or competing interests.

Clinical trial number Clinical trial number: not applicable.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Academy AD (2025) Qwen2-VL-72B: Vision-language model for multimodal understanding. <https://huggingface.co/alibaba/Qwen2-VL-72B>
- AI M (2023) CodeLLaMa-7B-instruct. <https://huggingface.co/meta/code-llama-7b-instruct>. Accessed 6 Mar 2025
- AlibabaDAMOAcademy (2025a) Qwen2-vL-7B: A compact multimodal model for code interpretation. <https://huggingface.co/alibaba/Qwen2-VL-7B>
- AlibabaDAMOAcademy (2025b) Qwen 2.5-72B: Refining multilingual and instruction following. <https://huggingface.co/alibaba/Qwen2.5-72B>
- AlibabaDAMOAcademy (2025c) Qwen2.5-coder-32B: A specialized model for algorithmic code generation. <https://huggingface.co/alibaba/Qwen2.5-Coder-32B>

- Anthropic (2025) Claude-3.7-Sonnet: High-performance code generation and debugging. <https://huggingface.co/anthropic/Claude-3.7-Sonnet>
- Arakelyan S, Das RJ, Mao Y, Ren X (2023) Exploring distributional shifts in large language models for code analysis. In: Conference on empirical methods in natural language processing. <https://doi.org/10.48550/arXiv.2303.09128>
- Austin J, Odena A, Nye M, Bosma M, Michalewski H, Dohan D, Jiang E, Cai C, Terry M, Le Q et al (2021) Program synthesis with large language models. [arXiv:2108.07732](https://arxiv.org/abs/2108.07732)
- Chen H, Chen D, Yang Y, Xu L, Gao L, Zhou M, Hu C, Li L (2025) Arkanalyzer: The static analysis framework for openharmony. [arXiv:2501.05798](https://arxiv.org/abs/2501.05798)
- Contributors HFM (2023a) DeepSeek-Coder-1.3B-Base. <https://huggingface.co/deepseek-coder-1.3b-base>. Accessed 6 Mar 2025
- Contributors HFM (2023b) DeepSeek-Coder-1.3B-Typescript. <https://huggingface.co/deepseek-coder-1.3b-typescript>. Accessed 6 Mar 2025
- Contributors HFM (2023c) Granite-3B-Code-Instruct-2K. <https://huggingface.co/granite-3b-code-instruct-2k>. Accessed 6 Mar 2025
- Contributors HFM (2023d) Granite-8B-Code-Base-4k-Output. <https://huggingface.co/granite-8b-code-base-4k-output>. Accessed 6 March 2025
- Contributors HFM (2023e) Viper-coder-hybridmini-v1.3. <https://huggingface.co/vipercoder-hybridmini-v1.3>. Accessed 6 Mar 2025
- DeepSeekAI (2024) DeepSeek-V3 technical report. <https://huggingface.co/deepseek-ai/DeepSeek-V3>
- DeepSeekAI (2025) DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. <https://huggingface.co/deepseek-ai/DeepSeek-R1>
- GoogleDeepMind (2025a) Google Gemini 2.0 flash: Optimizing AI for real-time applications. <https://huggingface.co/google/Gemini-2-Flash>
- GoogleDeepMind (2025b) Google Gemini 2.0 Pro: Advancements in multimodal AI. <https://huggingface.co/google/Gemini-2-Pro>
- Hendrycks D, Burns C, Kadavath S, Arora A, Basart S, Tang D, Song D, Steinhardt J (2021) Measuring coding challenge competence with apps. [arXiv:2105.09938](https://arxiv.org/abs/2105.09938)
- Hu X, Kuang K, Sun J, Yang H, Wu F (2024) Leveraging print debugging to improve code generation in large language models. <https://doi.org/10.48550/arXiv.2401.05319>
- Husain H, Wu HH, Gazit T, Allamanis M, Brockschmidt M (2019) Codesearchnet challenge: Evaluating the state of semantic code search. [arXiv:1909.09436](https://arxiv.org/abs/1909.09436)
- Jiang J, Wang F, Shen J, Kim S, Kim S (2024) A survey on large language models for code generation. [arXiv:2406.00515](https://arxiv.org/abs/2406.00515)
- Jiang X, Dong Y, Wang L, Shang Q, Li G (2023) Self-planning code generation with large language models. *ACM Trans Softw Eng Methodol*. <https://doi.org/10.1145/3672456>
- Kulal S, Saad F, Singh R, Chidambaram V, Kohli P (2019) SPoC: Search-based pseudocode to code. In: *Advances in neural information processing systems*, pp 11906–11917
- Li KS, Yuan Y (2024) Large language models as test case generators: Performance evaluation and enhancement. <https://doi.org/10.48550/arXiv.2404.13340>
- Li L, Gao X, Sun H, Hu C, Sun X, Wang H, Cai H, Su T, Luo X, Bissyande T et al (2023) Software engineering for openharmony: A research roadmap. *ACM Comput Surv*
- Lin F, Kim DJ, Chen TH (2024) Soen-101: Code generation by emulating software process models using large language model agents
- Lin Z, Zhou M, Ma W, Chen C, Yang Y, Wang J, Hu C, Li L (2025) HapRepair: Learn to repair openharmony apps. In: *Proceedings of the 33rd ACM international conference on the foundations of software engineering*, pp 319–330
- Lu S, Guo D, Ren S, Wang J, Tang D, Duan N, Zhou M (2021) Codexglue: A benchmark dataset and open challenge for code intelligence. [arXiv:2102.04664](https://arxiv.org/abs/2102.04664)
- MetaAI (2025a) Meta Llama 3.1-405B: Scaling instruction-tuned LLMs. <https://huggingface.co/meta-llama/Meta-Llama-3-405B>
- MetaAI (2025b) Meta Llama 3.1-70B: Advancements in instruction following. <https://huggingface.co/meta-llama/Meta-Llama-3-70B>
- MistralAI (2025) MistralAI codestral-2501: High-volume code generation with structural limitations. <https://huggingface.co/mistralai/Codestral-2501>
- OpenAI (2023) GPT-4: Advancements and limitations in code generation. <https://huggingface.co/openai/GPT-4>
- OpenAI (2025) OpenAI o3-mini: Efficient language and code model. <https://huggingface.co/openai/O3-Mini>
- Weyssow M, Zhou X, Kim K, Lo D, Sahraoui H (2023) Exploring parameter-efficient fine-tuning techniques for code generation with large language models. *ACM Trans Softw Eng Methodol*. <https://doi.org/10.48550/arXiv.2308.10462>

- xAI (2025) Grok-2-1212: Advancements in transformer architectures for logical reasoning. <https://huggingface.co/xai/Grok-2-1212>
- Yenduri G, Ramalingam M, Selvi GC, Supriya Y, Srivastava G, Maddikunta PKR, Raj GD, Jhaveri RH, Prabadevi B, Wang W et al (2024) GPT (generative pre-trained transformer)-a comprehensive review on enabling technologies, potential applications, emerging challenges, and future directions. IEEE Access 12:54608–54649
- Yeo S, Ma Y, Kim SC, Jun H, Kim T (2024) Framework for evaluating code generation ability of large language models. ETRI J. <https://doi.org/10.4218/etrij.2023-0357>
- Yin P, Neubig G (2018) Mining contextualized translation examples for model translation. In: Proceedings of the 2018 conference on empirical methods in natural language processing, pp 1387–1392
- Zhang S, Chen Z, Shen Y, Ding M, Tenenbaum J, Gan C (2023) Planning with large language models for code generation. In: International Conference on Learning Representations. <https://doi.org/10.48550/arXiv.2303.05510>
- Zhou B, Shi J, Wang Y, Li L, Li TO, Yu H, Zhu Z (2025) Porting software libraries to OpenHarmony: Transitioning from TypeScript or JavaScript to ArkTS. *Proceed ACM Softw Eng 2(ISSTA)*:1445–1466
- Zhu Y, Li J, Li G, Zhao Y, Li J, Jin Z, Mei H (2023) Hot or cold? adaptive temperature sampling for code generation with large language models. In: AAAI Conference on artificial intelligence. <https://doi.org/10.1609/aaai.v38i1.27798>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Ekin Can Erkuş¹  · Cansen Çağlayan Yılmaz¹ · Mustafa Umut Demirezen¹ 

✉ Ekin Can Erkuş
ekin.erkus@newcastle.ac.uk; ekincanerkus@hotmail.com

Cansen Çağlayan Yılmaz
cansen.caglayan@huawei.com

Mustafa Umut Demirezen
umut.demirezen@huawei.com; umut@demirezen.tech

¹ MSDC Department DC, Huawei Türkiye R&D Center, İstanbul, Türkiye